AI Talent Hub ИТМО

Практикум по исследовательской и разработческой деятельности в ML.

Занятие 1.

1. Знакомство

Преподаватель:
Бойматов Юсуфджон Шухратович,
Руководитель группы ML в RWB

Для кого этот курс?

- ML Engineer
- Data Engineer
- Data Analyst
- AI Product Developer

2. План курса

Блок 1: R&D-мышление и постановка эксперимента

Блок 2: Инструменты воспроизводимости и эксперимент-менеджмент

Блок 3: Улучшение модели как исследование

Блок 4: Диагностика и надежность

Блок 5: Коммуникация результатов и “handoff”

3. Что такое R&D

- **R&D** — это Research and Development, то есть исследования и разработка.

  Простыми словами:

- **Research** — изучаем проблему, проверяем гипотезы, ищем, что вообще может сработать.

- **Development** — превращаем найденную идею в рабочее решение, которое можно использовать на практике.

Разница с обычной разработкой:

- в обычной разработке чаще уже понятно, что строить

- в R&D сначала непонятно, сработает ли идея вообще

4. Почему это важно

Например - часто модели работают только “на бумаге”:

- неправильный split данных
- data leakage
- неправильная метрика

Пример:

5. Пример ML R&D

Задача:
классификация отзывов.

$$f(x) = \text{LogisticRegression}(TFIDF(x))$$

Baseline:
TF-IDF + Logistic Regression
F1 = 0.78

Гипотеза:

Контекстная модель (BERT) лучше понимает текст.

Эксперимент:

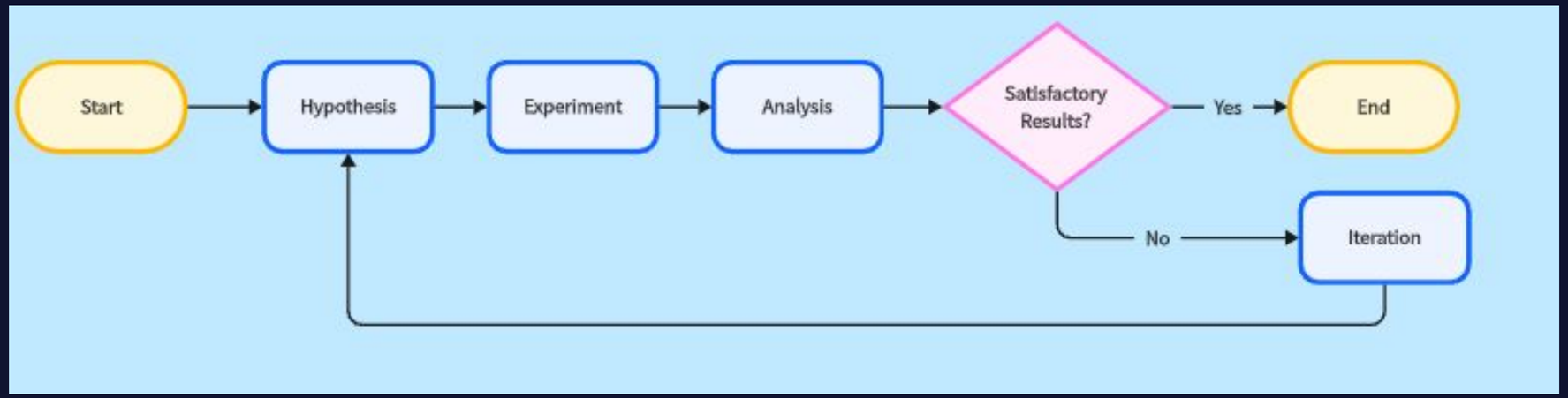
BERT fine-tuning

$$f(x) = BERT(x)$$

F1 = 0.84

Вывод:

модель улучшилась.



6. Гипотеза

Плохая гипотеза:

Попробуем другую модель.

Хорошая гипотеза:

Добавление temporal features улучшит AUC, потому что поведение пользователей зависит от времени.

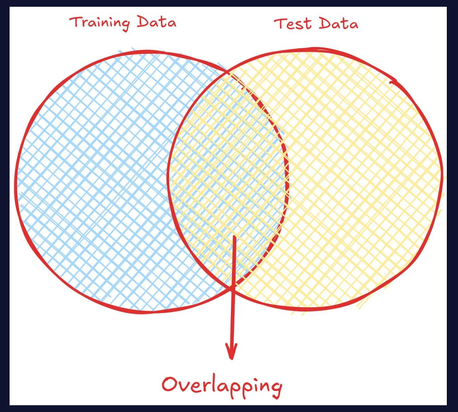
“Хорошая” гипотеза обычно отвечает в том числе критериям**:**

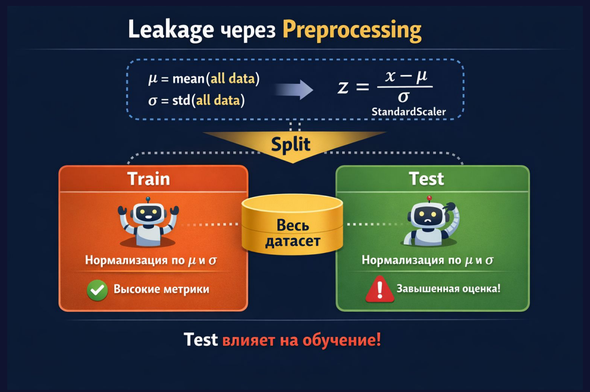
- *Реализуема*
- *проверяема (имеет критерий принятия)*
- *имеет понятный механизм (интерпретируема, объяснима)*
- *Обоснована*

7. Частая проблема ML

Data leakage:

модель использует информацию которая недоступна в момент предсказания





1. В признаки попал таргет или его кусок

Например, предсказываем, вернёт ли клиент кредит, а в признаки случайно попал столбец вроде days_overdue_after_loan.

Это информация из будущего, ее во время предсказания не будет.

2. Неправильный train/test split

Например, один и тот же пользователь есть и в train, и в test.

Тогда модель частично “узнаёт” объект заранее.

3. Препроцессинг сделан на всём датасете до split

Например, нормализацию, PCA, отбор признаков или target encoding сделали сразу на всём датасете, а потом разделили на train/test.

Тогда test уже повлиял на обучение.

4. Временная утечка

В задачах по времени нельзя использовать будущие данные для предсказания прошлого.

Например, хотим предсказывать продажи в январе, но в признаки попали агрегаты из февраля.

8. Metric Hacking

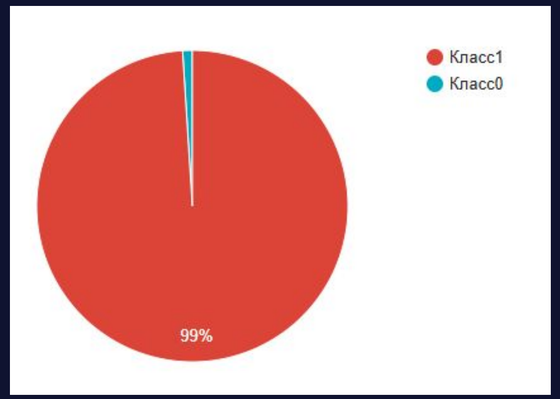
Пример:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FN + FP}$$

Датасет:

class0 = 99%

class1 = 1%



Почему это происходит

1. выбрали не ту метрику
2. слишком долго тюнили модель под один validation/test
3. начали принимать решения только по цифрам, игнорируя смысл
4. метрика не коррелирует с бизнес-показателями
5. нашли способ “обмануть” метрику без реального улучшения

Закон **Гудхарта**:

“Когда мера становится целью, она перестаёт быть хорошей мерой.”

Как с этим бороться

1. выбирать метрику, связанную с реальной целью
2. смотреть несколько метрик, а не одну
3. держать честный test set
4. не тюнить бесконечно по одному и тому же validation
5. делать error analysis
6. проверять результат на данных, близких к реальному сценарию
7. оценивать не только offline-метрики, но и продуктовый смысл

9. Train / Validation / Test

Разделение данных:

train

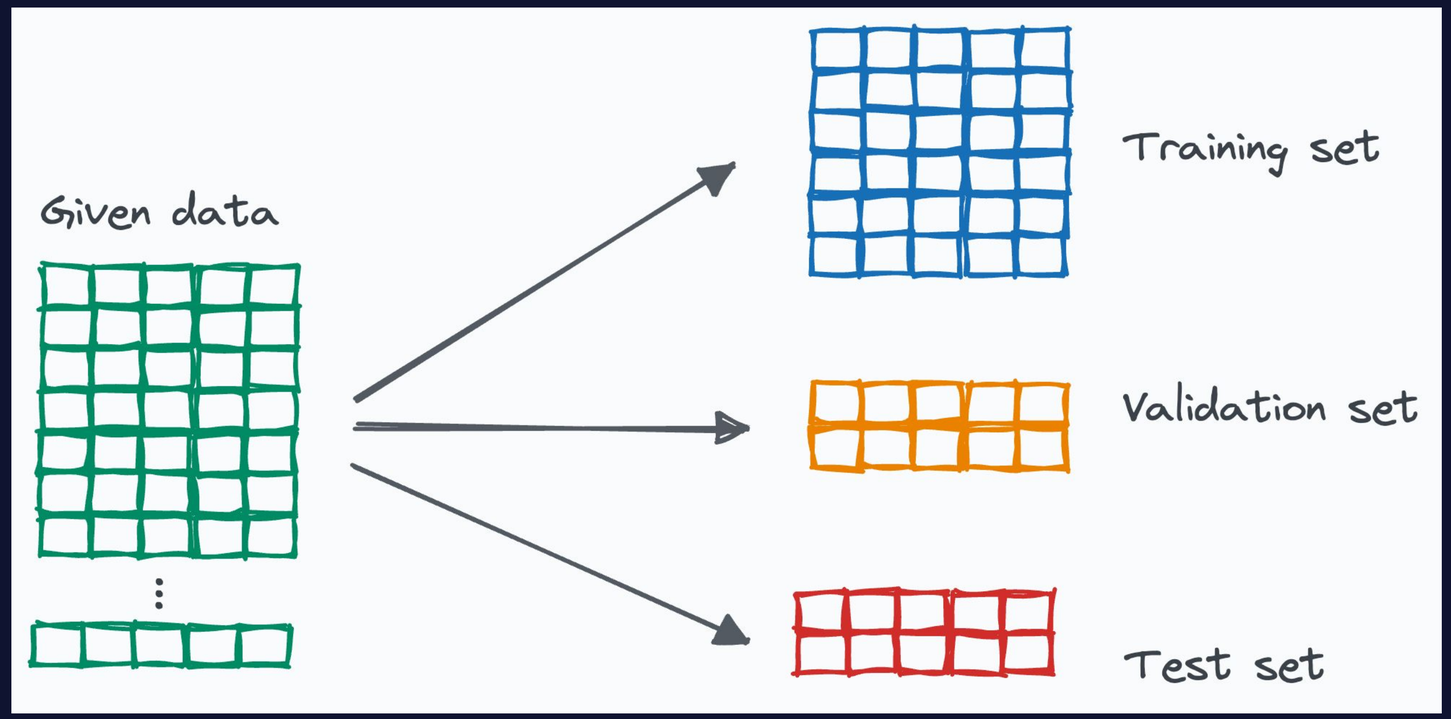
validation

test

train — оптимизация параметров

$$\theta = argmin\hat{R}_{train}$$

test— выбор гиперпараметров



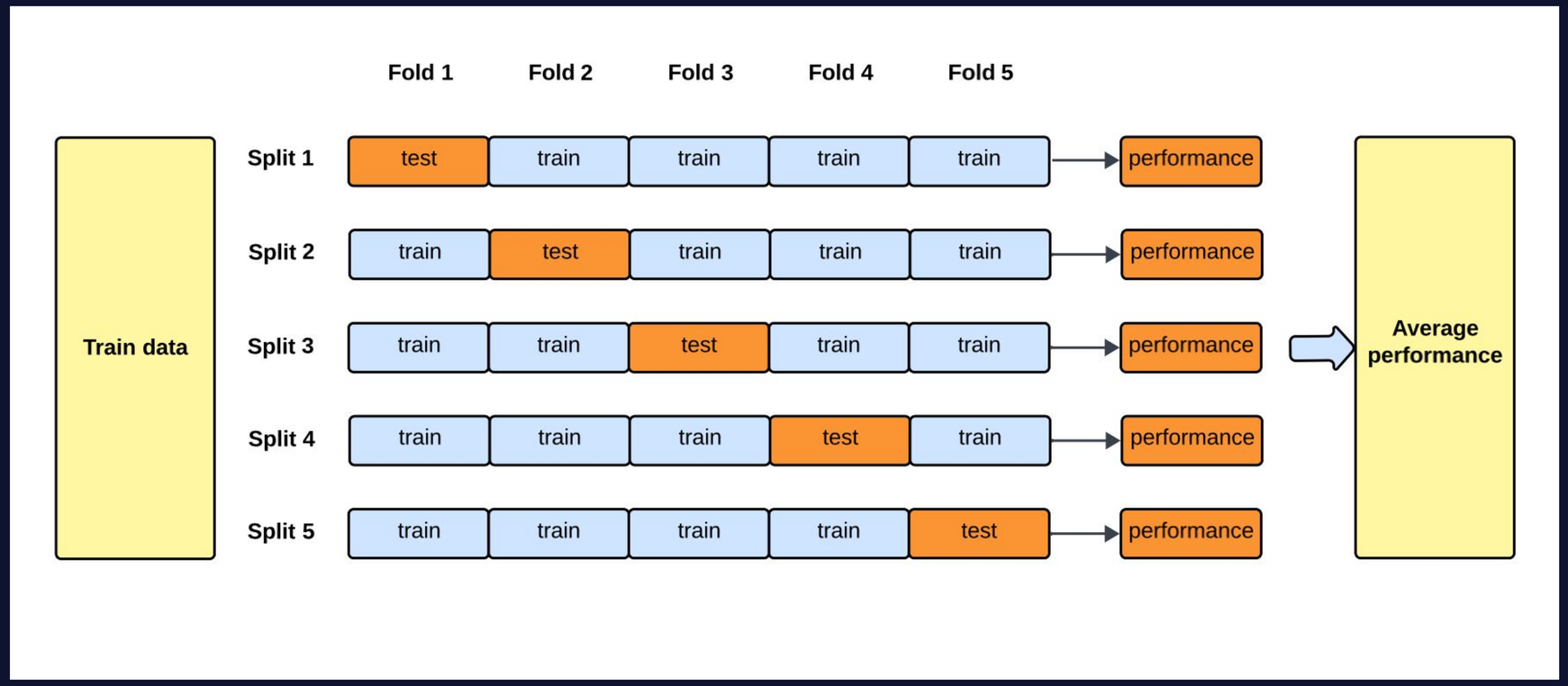
10. Cross Validation

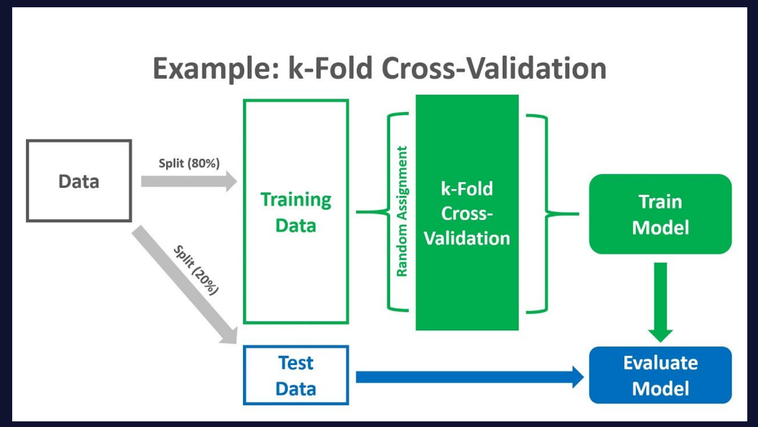
Идея:

Dataset делится на k частей:

Метрика при этом:

$$\hat{R} = \frac{1}{k}\sum_{i=1}^{k} R_i$$





11. Stratified Split

Идея:

Сохраняем распределение классов.

Если:

$$P(y=1)=p$$

То:

$$P_{train}(y=1) \approx P_{test}(y=1)$$

12. Bootstrap

Идея:

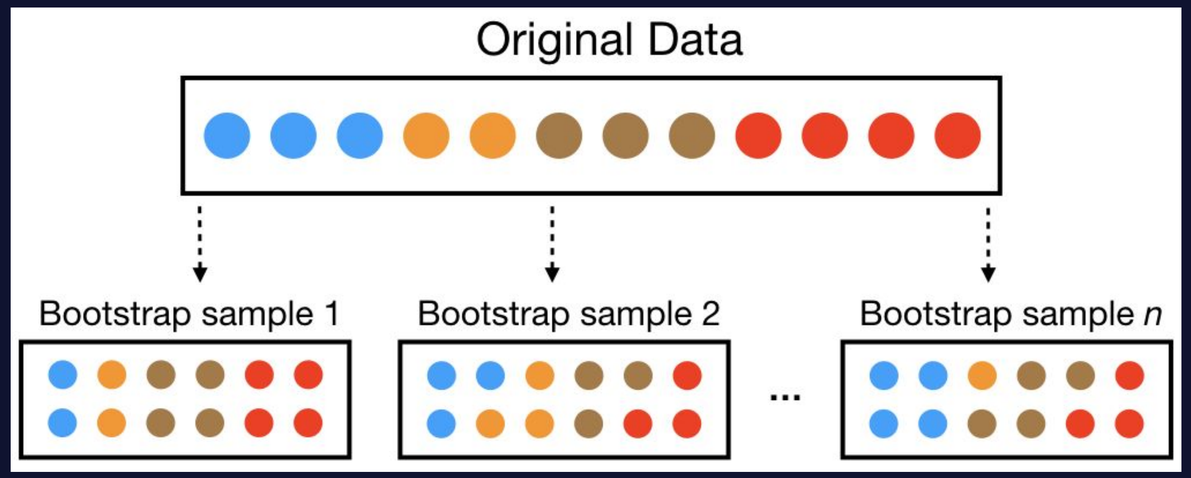
1. *выбираем случайную подвыборку с возвращением*
2. *считаем метрику*
3. *повторяем B раз:*

$$\{\hat{R}_1, \hat{R}_2, ..., \hat{R}_B\}$$

Зачем это нужно

Чаще всего bootstrap используют, чтобы оценить:

1. доверительный интервал метрики;
2. насколько стабильна оценка;
3. стандартную ошибку;
4. различие между моделями.



12. Bootstrap

$$\mathrm{Var}_{\mathrm{boot}}(T) = \frac{1}{B-1}\sum_{b=1}^{B}\left(T^{(b)} - \bar{T}_{\mathrm{boot}}\right)^2$$

$$CI_{1-\alpha} = \left[Q_{\alpha/2}(T^{(1)},\ldots,T^{(B)}),\ Q_{1-\alpha/2}(T^{(1)},\ldots,T^{(B)})\right]$$

$$CI_{0.95} = [Q_{0.025}, Q_{0.975}]$$

$$P(\Delta > 0) \approx \frac{1}{B}\sum_{b=1}^{B}\mathbf{1}(\Delta^{(b)} > 0)$$

- $\mathrm{Var}_{\mathrm{boot}}(T)$ — **бутстрап-оценка дисперсии** статистики T
- T — статистика, которую мы изучаем
  - например: accuracy, F1, AUC, mean, median
- B — число бутстрап-выборок
  - например, $B = 1000$
- b — номер конкретной бутстрап-выборки, где

$$b = 1, 2, \ldots, B$$

- $T^{(b)}$ — значение статистики T, посчитанное на b-й бутстрап-выборке
- $\bar{T}_{\mathrm{boot}}$ — среднее значение статистики по всем бутстрап-выборкам:

$$\bar{T}_{\mathrm{boot}} = \frac{1}{B}\sum_{b=1}^{B} T^{(b)}$$

- $\sum_{b=1}^{B}$ — суммирование по всем бутстрап-повторам
- $\left(T^{(b)} - \bar{T}_{\mathrm{boot}}\right)^2$ — квадрат отклонения очередной бутстрап-оценки от среднего
- $\frac{1}{B-1}$ — нормировка, как в обычной выборочной дисперсии

QA